
EA PRACTICAL ASSIGNMENT REPORT

Sheng Yi
s4196864
s.yi.2@umail.leidenuniv.nl

Yutao Liu
s4213211
y.liu.67@umail.leidenuniv.nl

December 24, 2024

1 Introduction

We implemented the classic genetic algorithm (GA), specifically a binary-coded GA that includes selection, crossover, and mutation operations. During the hyperparameter tuning process, we incorporated the NSGA-II algorithm [1], a multi-objective optimization approach, to select solutions that consider the optimal outcomes for two objectives.

The tuning process quickly gets optimal hyperparameters as long as an appropriate parameter range is set. When the total tuning budget increases to 5,000,000, the granularity of parameter selection can be reduced, allowing for more precise identification of suitable parameters. To explore factors beyond hyperparameters, we also considered different selection methods, crossover methods, and mutation methods as tuning options, enabling greater possibilities for exploration.

The ES (evolution strategy) part turned out to be more complex than anticipated because it required determining the type of mutation. The mutation type largely dictates the appropriate selection and recombination functions, making it less compatible compared to the GA part. Based on the course content, correlation mutation is more flexible for exploration across different problems, which is why we ultimately chose it as our mutation method. However, we later made adjustments to the recombination process to enhance the exploratory capabilities of the ES component.

2 Algorithms

2.1 Implementation in Genetic Algorithm

The algorithm has two types of selection mechanisms to choose parents for the next generation. The first is tournament selection, where a random subset of individuals is chosen, and the individual with the highest fitness within the subset is selected. The second mechanism is proportional selection, also known as roulette wheel selection, which normalizes the fitness of all individuals into probabilities. This method gives individuals with higher fitness a greater chance of being chosen as parents.

For crossover operations, the algorithm incorporates two approaches. In one-point crossover, a single point is chosen on the parent chromosomes, and all bits to the right of this point are swapped between the two parents. Alternatively, uniform crossover is a genetic algorithm operation where each gene in the offspring is independently chosen from either parent with equal probability.

Mutation is implemented through two methods to enhance the search process. The standard bit mutation flips each bit in an individual's chromosome with a probability equal to the mutation rate, which is effective for fine-tuning solutions and performing local searches. Multi-bit mutation, on the other hand, flips a number of bits determined by the mutation rate, encouraging broader exploration of the search space and increasing diversity within the population.

After each generation, the offspring population completely replaces the current population, ensuring that only the new generation is retained for subsequent iterations. The algorithm continues to evolve the population until a pre-defined maximum number of fitness evaluations is reached, providing a clear and consistent termination criterion.

Method	Crossover	Mutation	Selection
Type 1	Uniform Crossover	Standard Mutation	Tournament Selection
Description	Genes chosen randomly from both parents.	Each bit in chromosome is flipped.	Random subset, best fitness chosen.
Advantage	High diversity, good for complex problems.	Local search.	Simple, ensures diversity.
Type 2	Single Point Crossover	Multi-bit Mutation	Proportional Selection
Description	One crossover point, genes swapped after it.	Multiple bits flipped at once.	Selection based on fitness probability.
Advantage	Simple, good for strongly related genes.	Encourages broad exploration, increases diversity.	Faster convergence, good for larger populations.

Table 1: Comparison of Crossover, Mutation, and Selection Methods

Table 1 is a comparison of the methods we used. And we use pseudo-code (Algorithm 1) to explain our GA:

Algorithm 1: Genetic Algorithm (GA)

Input: Population size μ , Mutation rate p_m , Crossover probability p_c , Maximum evaluations B

Output: Best solution x^* and its fitness f^*

- 1 **Initialization:** Randomly initialize μ individuals;
 - 2 **Evaluate:** Compute fitness for all individuals;
 - 3 **while** $evaluations < B$ **do**
 - 4 **Selection:** Use proportional or tournament selection to choose parents;
 - 5 **Crossover:** Apply uniform crossover on selected parents with probability p_c to generate offspring;
 - 6 **Mutation:** Mutate offspring using bit-flip mutation with rate p_m ;
 - 7 **Evaluate:** Compute fitness for offspring;
 - 8 **Replacement:** Replace the current population with offspring;
 - 9 **Return:** x^* and f^* ;
-

2.2 Implementation in Evolution Strategy

To address the F23 problem (Katsuura Problem) from the Black-Box Optimization Benchmarking (BBOB) problem set, we implemented a correlation mutation based ES model to meet the requirements of the provided questions. The implementation includes several key features and processes.

For recombination, three types are supported. In discrete recombination, each gene of the offspring is randomly selected from one of the two parents, guided by randomly created masks that determine which parent's gene will be inherited. Alternatively, in global intermediary recombination, all parents contribute to the offspring creation through a weighted averaging process, where the weights are randomly generated and normalized to sum to one. This process applies to both object parameters (solution values) and strategy parameters (step sizes and rotation angles). In the variation of global intermediary recombination, we no longer assign random weights but instead choose the average, allowing all offspring to benefit uniformly from the parents.

The mutation process uses correlated mutation techniques. Step sizes are mutated using global and local random factors adjusted by learning rates (τ and τ'), while rotation angles are perturbed with Gaussian noise. This mutation process incorporates covariance matrices to introduce correlations among variables.

For selection, two strategies are available. The (μ, λ) strategy selects the top μ offspring from the offspring population, while the $(\mu + \lambda)$ strategy selects the top μ individuals from a combined pool of parents and offspring.

Fitness evaluation is performed using a problem-specific fitness function. Offspring are assessed, and the population is updated based on the chosen selection strategy.

Finally, the algorithm terminates when the minimum number of fitness evaluations is reached, ensuring that computational resources are managed effectively.

The pseudo-code for Evolution Strategy is shown in Algorithm 2:

Algorithm 2: Evolution Strategy (ES)**Input:** Initial population size μ , offspring size λ , Mutation step size σ , Maximum evaluations B **Output:** Best solution x^* and its fitness f^*

```

1 Initialization: Randomly initialize  $\mu$  individuals with step sizes and rotation angles;
2 Evaluate: Compute fitness for all individuals;
3 while evaluations <  $B$  do
4   Recombination: Apply discrete or global intermediary recombination to generate  $\lambda$  offspring;
5   Mutation: Perform correlated mutation on offspring using step sizes and rotation angles;
6   Evaluate: Compute fitness for offspring;
7   Selection: if  $(\mu, \lambda)$  strategy then
8     | Select the best  $\mu$  offspring based on fitness;
9   else if  $(\mu + \lambda)$  strategy then
10    | Select the best  $\mu$  individuals from combined parent and offspring populations;
11 Return:  $x^*$  and  $f^*$ ;

```

3 Hyper-parameter tuning

We performed hyperparameter tuning for a GA using a comprehensive and structured approach to identify the optimal configuration. The hyperparameters explored included population size, mutation rate (ranging from 0.01 to 0.1), and crossover rate. We also examined two mutation types (standard mutation and multi-bit mutation) and two crossover types (single-point and uniform crossover). Selection methods included proportional selection and tournament selection, allowing us to test how these choices influenced algorithm performance. We can see the specific parameter selection in Table 2.

Hyperparameter	Values
Population size	5, 8, 10, 20, 25, 30
Mutation rate	0.01, 0.015, 0.02, 0.03, 0.05, 0.1
Crossover rate	0.5, 0.6, 0.7, 0.75, 0.8
Mutation type	Standard mutation, Multi-bit mutation
Crossover type	Single-point crossover, Uniform crossover
Selection method	Proportional selection, Tournament selection

Table 2: Hyperparameters and their respective values for GA tuning.

To evaluate the configurations, we selected two benchmark problems: F18 (the LABS problem) and F23 (the N-Queens problem). For each hyperparameter configuration, the algorithm was run 20 times independently on each problem to ensure robust and unbiased performance measurement. Average fitness scores across the runs were recorded to evaluate each configuration's effectiveness.

The search process was iterative, starting with a predefined evaluation budget. Each configuration was tested on the benchmark problems, and the number of evaluations performed for each problem was dynamically adjusted based on the remaining budget. Once all evaluations for a configuration were complete, its performance metrics (average fitness for F18 and F23) were stored for further analysis. This iterative process continued until the entire budget was exhausted.

To identify the best configurations, we performed Pareto front analysis to balance performance across both problems. By applying Pareto optimality principles, we identified a set of non-dominated solutions that represented the best trade-offs between the objectives of maximizing fitness on F18 and F23. These non-dominated configurations were analyzed further to select the best compromise solution.

The selection of the final configuration involved calculating the normalized Euclidean distance of each Pareto-optimal solution to the ideal point, defined as the maximum fitness values for both objectives. The configuration with the smallest distance to the ideal point was selected as the optimal solution, balancing performance across both benchmark problems.

This process ensured a thorough exploration of the hyperparameter space while maintaining robustness through multiple independent runs and careful consideration of the trade-offs between competing objectives. The result was an optimal configuration that demonstrated high performance across diverse evaluation criteria.

We also design the pseudo-code of the hyperparameter tuning process in Algorithm 3:

Algorithm 3: Hyperparameter Tuning Process

Input: Hyperparameter space $\mathcal{H}$: Population size, Mutation rate, Crossover rate

Problems P : $\{F18, F23\}$

Budget B : Total number of evaluations

Repetitions R : Number of independent runs per setting

Output: Best hyperparameter configuration h^*

```

1 Initialization: Set remaining budget  $B_{\text{rest}} = B$ , initialize results list  $\mathcal{R} \leftarrow \{\}$ ;
2 foreach  $h \in \mathcal{H}$  do
3   foreach  $P_i \in P$  do
4     Set evaluation limit:  $E_i = \min(B_{\text{rest}}, \text{evaluations per run})$ ;
5     Run Algorithm: Execute  $R$  independent runs of the algorithm on  $P_i$  with  $h$ ;
6     Record Result: Compute average performance  $\overline{f}_i$  over  $R$  runs;
7     Update  $B_{\text{rest}} \leftarrow B_{\text{rest}} - (R \times E_i)$ ;
8     Reset  $P_i$ ;
9     if  $B_{\text{rest}} \leq 0$  then
10      Exit Loops;
11   Store Configuration Result: Append  $\{h, \overline{f}_{F18}, \overline{f}_{F23}\}$  to  $\mathcal{R}$ ;
12 Pareto Front Analysis:
13 Identify non-dominated configurations from  $\mathcal{R}$  using Pareto optimality;
14 Select Best Compromise Solution:
15 foreach Pareto-optimal solution  $r \in \mathcal{R}$  do
16   Compute normalized Euclidean distance to the ideal point  $(\max \overline{f}_{F18}, \max \overline{f}_{F23})$ ;
17 Select the solution  $h^*$  with the smallest distance;
18 Return: Best configuration  $h^*$ ;
```

The debugging of the ES (Evolution Strategy) part was done without relying on tuning code, focusing instead on manually adjusting the selection methods and recombination strategies. Specifically, the process involved experimenting with different combinations of the selection strategies, such as (μ, λ) and $(\mu + \lambda)$, varying the values of μ (the number of parents) and λ (the number of offspring), and testing different recombination methods. The goal was to find the most effective combination by iteratively testing these configurations, considering how each affects population diversity and convergence speed. Ultimately, this approach aimed to balance exploration and exploitation to achieve the best performance for the optimization task. We also design the pseudo-code of the EA debugging process with Algorithm 4:

Algorithm 4: Evolution Strategy (ES) Debugging

Input: Initial population P_0 , Selection strategies (μ, λ) or $(\mu + \lambda)$, Recombination methods

Output: Best configuration found

```

1 Initialize: Initialize population  $P_0$  with random individuals for each combination of selection strategy and recombination method;
2 foreach selection strategy  $(\mu, \lambda)$  or  $(\mu + \lambda)$  do
3   foreach recombination method do
4     foreach  $\mu, \lambda$  configuration do
5       Select parents from current population  $P_t$ ;
6       Generate offspring  $O_t$  using recombination methods;
7       Evaluate fitness of parents and offspring;
8       Select next generation  $P_{t+1}$  based on selected strategy;
9       if convergence criteria met or max generations reached then
10        Break;
11     if best performance achieved then
12       Store current configuration;
13 Return: The best configuration found;
```

4 Experimental Results

4.1 Genetic Algorithm

From Table 3, the differences between these two problems can be observed. The N-Queens problem has a theoretical upper limit for fitness values at N , while LABS requires continuous exploration to approximate the optimum. During the tuning process, the optimal fitness values for LABS are relatively balanced, with less disparity between the best, worst, and median individuals. In contrast, the N-Queens problem often generates negative fitness values for the worst and best individuals due to the large penalty function, and negative solutions are frequently encountered during tuning. The final optimal hyperparameters, as shown in Table 4, achieve a fitness value of 4.01 for LABS and 6 for the 7-Queens problem simultaneously.

Problems	DIM	funcId	budget	best reached	mean reached	worst recorded	worst reached
F18	49	23	5000	7	-199.22	-679	-450
F23	50	18	5000	5.76	3.12	0.36	1.04

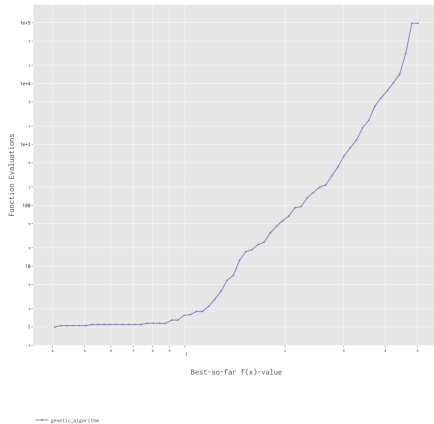
Table 3: Tuning Data with Comparison Problems F18 and F23

Key	Value
params	[5, 0.01, 0.5, 'standard', 'uniform', 'proportional', 0]
F18	4.008076728810781
F23	6.05

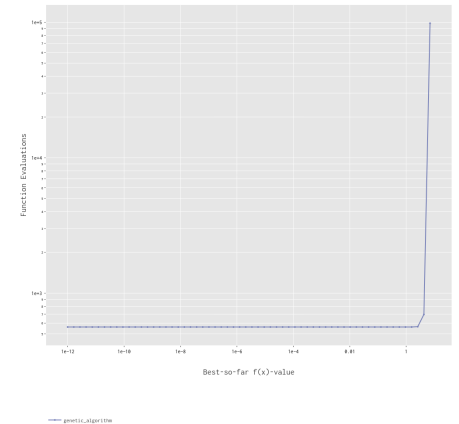
Table 4: Parameter and Function Values

From IOH, we know that the approximated Area Under the Curve of F18 displayed single-function ECDF is: 0.87026 while the approximated Area Under the Curve of F23 displayed single-function ECDF is: 0.548853.

We used the best parameters that found in tuning process to run the result. The ERT For F18 is shown in Figure 1.a, their curve starts with a steep improvement in the objective value for a relatively small number of function evaluations. Progress slows down as the number of evaluations increases, showing diminishing returns. Eventually, the curve plateaus, indicating no significant improvement in the best objective value despite further evaluations. The logarithmic y-axis reveals that the algorithm initially requires fewer evaluations for large improvements but later demands exponentially more evaluations for smaller gains. As for F23, The ERT Curve in Figure 1.b shows the relationship between the best-so-far $f(x)$ -value and the number of function evaluations. The graph remains almost flat for very small $f(x)$ -values (near 10^{-12} to 10^{-4}), indicating minimal improvement in early evaluations. However, as $f(x)$ approaches 1, the number of function evaluations increases sharply, suggesting a steep cost for further optimization.



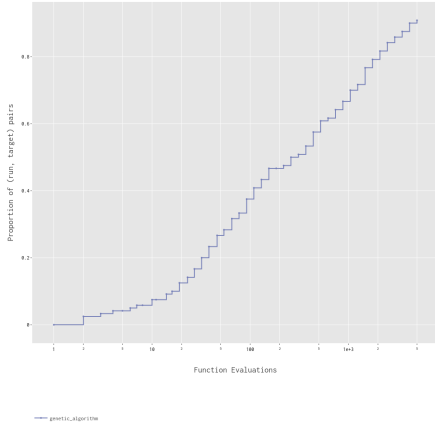
(a) Genetic Algorithm-ERF Curve (F18)



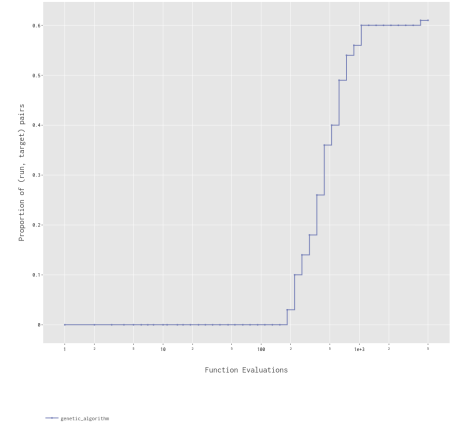
(b) Genetic Algorithm-ERF Curve (F23)

Figure 1: Comparison of Genetic Algorithm ERF Curves for F18 and F23

The ECDF Curve for F18 in Figure 2.a shows increasing effectiveness as the number of function evaluations grows. The final proportion of successful runs reaching the target value is close to 80%, which indicates reasonably good performance. The Early evaluations result in minimal success, likely because the algorithm is still exploring the solution space; However, The mid-stage growth reflects the exploitation phase, where the algorithm has identified promising regions of the search space. Finally, The late plateau indicates that further evaluations provide diminishing returns, as most runs have already achieved the target. As for F23, The ECDF Curve in Figure 2.b illustrates the cumulative success proportion as a function of evaluations. The curve starts at zero, showing no success initially, but then increases steadily after approximately 10^2 evaluations and eventually stabilizes around 0.6. This indicates that the genetic algorithm performs moderately well, achieving success in 60% of the runs.



(a) Genetic Algorithm-ECDF Curve (F18)



(b) Genetic Algorithm-ECDF Curve (F23)

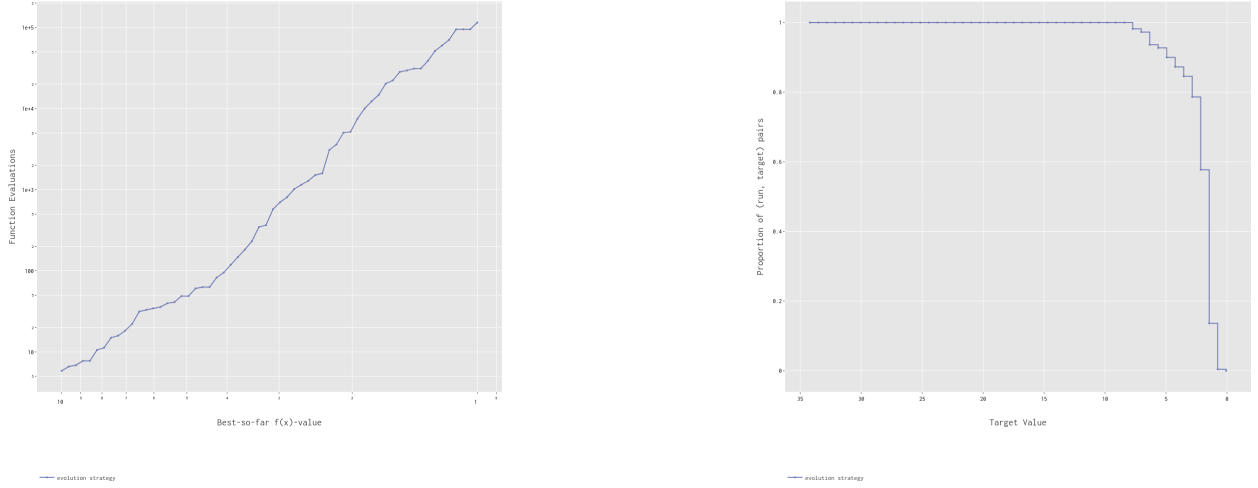
Figure 2: Comparison of Genetic Algorithm ECDF Curves for F18 and F23

4.2 Evolutuion Strategy

We can see the ERT and ECDF of ES algorithm with 15+100 selection in Figure3. The ERT curve of Evolution Strategy shows the improvement of the best-so-far objective value $f(x)$ as the number of function evaluations increases. The curve begins at higher $f(x)$ values and progressively moves toward lower values as evaluations increase. The decrease in $f(x)$ shows steady improvement throughout, with no visible plateau or stagnation. The logarithmic scale highlights that smaller improvements in $f(x)$ require increasingly more evaluations. The absence of a plateau indicates that the algorithm continues to make progress even with a large number of evaluations, potentially reaching near-optimal solutions.

The ECDF Curve of Evolution Strategy shows the proportion of successful runs achieving a given target value. The x-axis represents the target value, while the y-axis represents the cumulative proportion of successful runs. For target values around 35, the proportion of successful runs is close to 1, indicating all runs achieve these higher target values. The success proportion remains constant near 1 for higher target values but starts to drop as the target value decreases. A sharp decline in success occurs as the target values approach 0, suggesting that the algorithm struggles to achieve these smaller target values.

The algorithm performs well for less demanding targets (higher values) but finds it increasingly challenging to meet stricter criteria (lower target values).

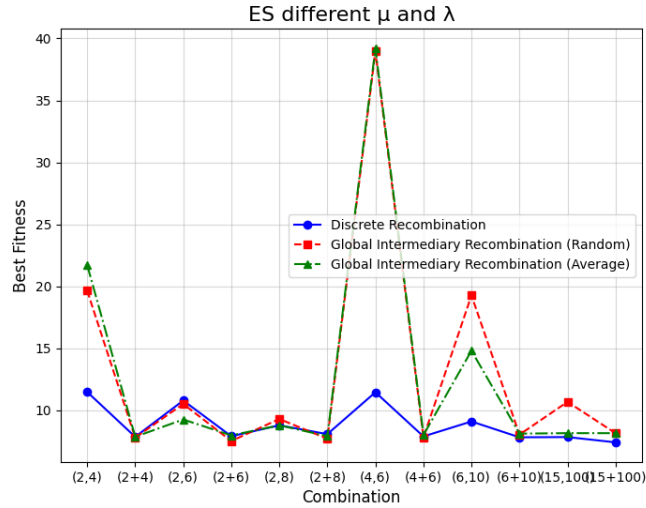


(a) Evolution Strategy- ERT Curve

(b) Evolution Strategy- ECDF Curve

Figure 3: Comparison of Evolution Strategy Curves (ERT and ECDF)

The debugging process of ES is illustrated in Figure 4, where it is evident that the $\mu + \lambda$ strategy achieves lower and more stable fitness values compared to μ, λ . This aligns with expectations, as under budget constraints, the $\mu + \lambda$ strategy effectively facilitates progress, while μ, λ may risk regression. In the final, based on the results, selecting discrete recombination with 15+100 generates our current best fitness value of 7.41. The final choice of 15+100 was inspired by the content of the slides, which allowed us to achieve a relatively ideal result.

Figure 4: Comparison of Different μ and λ Combination

5 Discussion and Conclusion

In the GA section, a smaller population size often leads to more thorough evolution. This might be because, under budget constraints, a higher number of iterations is slightly more critical than the population size. More generations increase the likelihood of finding a global optimal solution. When handling LABS and the N-Queens problem, a lower mutation rate (set to 0.01 in this experiment) also tends to yield better results. This might be because these problems do not require extensive exploration in the search space, and steady progress is more crucial. The combination of standard mutation, proportional selection, and uniform crossover produces better outcomes.

Notably, in the N-Queens problem, proportional selection outperforms tournament-k selection. This is likely because, in problems where fitness values vary significantly, proportional selection is more effective at identifying and propagating the best offspring. By consistently favoring individuals with higher fitness, proportional selection ensures that the search process remains focused on promising areas, avoiding the potential pitfalls of stochastic fluctuations that can occur with tournament-based methods in such contexts. This synergy of techniques highlights the importance of tailoring the algorithm's parameters and operators to the specific characteristics of the problem at hand.

In the ES section, different recombination methods exhibit varying levels of effectiveness. Compared to the two global intermediary recombination methods, discrete recombination demonstrates greater stability, ensuring that the variation in results remains minimal. This could be attributed to its inherent randomness, which helps maintain population diversity more effectively. Diversity, in turn, supports a broader exploration of the solution space, reducing the risk of premature convergence.

Additionally, the $\mu + \lambda$ strategy consistently achieves better fitness values than its counterpart μ, λ with the same numerical configuration. As discussed in the results section, the advantage lies in the $\mu + \lambda$ strategy's ability to avoid regression. By ensuring that no solution backslides, it guarantees steady progress within the given budget constraints.

Finally, the choice of the 15+100 configuration strikes an optimal balance between diversity and steady improvement. The relatively large λ ensures sufficient exploration, while the combination of discrete recombination and $\mu + \lambda$ allows for steady refinement of solutions. This enables the algorithm to efficiently and effectively achieve the desired results within a reasonable time. The integration of these elements shows the importance of tailoring strategies to leverage both population diversity and progression stability for complex optimization problems.

References

- [1] Kalyanmoy Deb, Arun Pratap, Satyen Agarwal, and Thais Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.